# Exploring the WLasso Approach to Regularization

Mariana Garcia Mejia

November 9, 2026

## 1 Reproducing WLasso on simulated data

The paper A variable selection approach for highly correlated predictors in high-dimensional genomic data proposes a new approach to perform regularization in data sets with highly correlated variables. When variables in a dataset are highly correlated, the **Irrepresentable Condition (IC)**, fails to hold. This means that non-active variables can be represented as a combination of active variables, and variable selection techniques like Lasso are unable to tell apart the real signals. For instance, if X1 and X2 can explain the outcome Y, and X3 can be represented in terms of X1 and X2, then X3 will line up with Y as well. The WLasso technique transforms the data, the design matrix in particular, to ensure that the Irrepresentable Condition holds.

The following bullet points highlight characteristics to keep in mind:

- High-dimensionality: p » n
- Observations are independent (Fish 1 info. tells me nothing about Fish 2, likely doesn't hold in the fish dataset).
- X is highly correlated and the IC doesn't hold.

### 1.1 Simulating the data

Section 2.1 of the paper describes a simulation approach that we attempt to reproduce in the first part of this write up. They simulate a dataset with $n = 50$, and $p = 500$ where 10 variables are active and the rest are non-active. For simplicity purposes, we take $p = 100$. Additionally, they use a pre-set correlation matrix with correlation between active variables (SS) $\alpha_1 = 0.3$, correlation between active and non-active variables (S $S^c$) $\alpha_2 = 0.5$, and correlation between non-active variables ($S^c$ $S^c$) $\alpha_3 = 0.7$. As shown below:

$$R = \begin{pmatrix} R_{SS} & R_{SS^c} \\ R_{S^cS} & R_{S^cS^c} \end{pmatrix} = \begin{pmatrix} 1 & 0.3 & \cdots & 0.3 & 0.5 & \cdots & 0.5 \\ 0.3 & 1 & \cdots & 0.3 & 0.5 & \cdots & 0.5 \\ \vdots & \vdots & \ddots & \vdots & \vdots & & \vdots \\ 0.3 & 0.3 & \cdots & 1 & 0.5 & \cdots & 0.5 \\ 0.5 & 0.5 & \cdots & 0.5 & 1 & 0.7 & \cdots \\ \vdots & \vdots & & \vdots & 0.7 & \ddots & \vdots \\ 0.5 & 0.5 & \cdots & 0.5 & \cdots & \cdots & 1 \end{pmatrix}$$

The paper describes they simulate independent Gaussian vectors with covariance matrix $R$. Hence we simulate $X_i \sim N(0, R)$ to form the design matrix $X$. Note that $\beta$ contains 10 active variables with value 2, the rest are non-active so the coefficients are zero. The active variables are in the 10 first entries of the $\beta$ matrix to match with the correlation matrix. We use the design matrix and the coefficients from above to predict:

$$y = X\beta + \epsilon$$

with

$$X_i \overset{\text{iid}}{\sim} N(0, R), \qquad \beta_j = \begin{cases} 2, & j \in S, \\ 0, & j \in S^c. \end{cases}$$

```
library(corrplot)
```

```
Warning: package 'corrplot' was built under R version 4.3.3
```

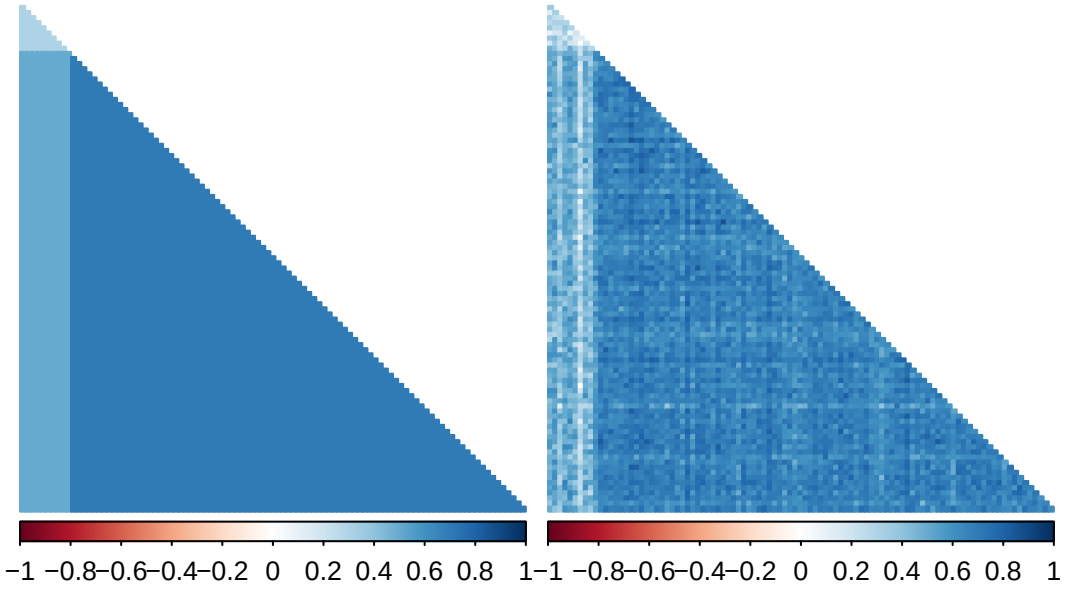
```
corrplot 0.95 loaded
```

```
R_empirical <- cor(X)

par(mfrow = c(1, 2))

# 1. Population correlation matrix
corrplot(
  R,
  method = "color",
  type = "lower",
  tl.pos = "n",
  diag = FALSE
)
```

```r
# 2. Empirical correlation matrix
corrplot(
  R_empirical,
  method = "color",
  type = "lower",
  tl.pos = "n",
  diag = FALSE
)
```




```r
par(mfrow = c(1, 1))
```

## 1.2 Checking the Irrepresentable Condition

First, we check that the Irrepresentable Condition does not hold. As previously mentioned, the Irrepresentable Condition (IC) is an important condition for the Lasso to consistently identify the true set of nonzero coefficients. It requires that the inactive predictors cannot be too strongly represented by the active predictors.

Let $S = \{j : \beta_j \neq 0\}$ denote the set of active predictors and $S^c$ the set of inactive predictors. The IC is defined as:

$$\left|X_{S^c}^T X_S (X_S^T X_S)^{-1} \operatorname{sign}(\beta_S)\right|_j \leq 1 - \gamma, \qquad \forall j \in S^c,$$

where $\gamma \in (0, 1]$.

Here:

- $X_S$ contains the columns of $X$ corresponding to active predictors.
- $X_{S^c}$ contains the columns corresponding to inactive predictors.
- $\operatorname{sign}(\beta_S)$ is the vector of signs of the nonzero coefficients.
- $\gamma$ represents a positive margin ensuring that the condition is strictly satisfied.

In our simulation, we have 10 active and 90 inactive predictors. Therefore, the expression produces a vector of 90 values, one for each inactive predictor.

The maximum IC value obtained from our simulated data was 1.6816, exceeding the required threshold of 1. Therefore, the Irrepresentable Condition is violated. Furthermore, 86 out of 90 inactive predictors (95.56%) have IC values greater than or equal to 1. This suggests that the correlation structure makes it difficult for the Lasso to distinguish relevant from irrelevant predictors, potentially compromising its variable selection performance.

## 1.3 Fitting the regular Lasso

Next, we fit the regular Lasso to have a baseline comparison for WLasso. The regular Lasso minimizes the MSE with an L1 penalty. This enforces sparsity more compared to for instance Ridge Regularization. WLasso benefits from this.

$$\hat{\beta}^{\text{Lasso}} = \arg\min_{\beta} \left\{ \|y - X\beta\|_2^2 + \lambda \|\beta\|_1 \right\},$$

where

$$\|\beta\|_1 = \sum_{j=1}^{p} |\beta_j|.$$

Table 1: Classical Lasso variable selection results on the simulated dataset.

| Metric | Value |
|---|---|
| True Positives (TP) | 10 |
| False Positives (FP) | 23 |
| False Negatives (FN) | 0 |
| Total Selected | 33 |

The Lasso identified 33 total variables: all 10 active variables correctly, and 23 additional non-active variables. These results illustrate the difficulty of variable selection in the presence of highly correlated predictors. Although Lasso successfully recovered all relevant variables, it failed to exclude many irrelevant ones.

## 1.4 Introducing WLasso

The objective of the whitening transformation is to remove correlations between predictors, addressing one of the main difficulties faced by classical Lasso when the Irrepresentable Condition (IC) is violated.

Let $R \in \mathbb{R}^{p \times p}$ denote the population correlation matrix of the predictors.

Since $R$ is symmetric and positive definite, we can apply eigendecomposition:

$$R = UDU^T,$$

where:

- $U$ is an orthogonal matrix whose columns are the eigenvectors of $R$, satisfying $U^TU = UU^T = I_p$.
- $D = \text{diag}(d_1, \dots, d_p)$ is a diagonal matrix containing the positive eigenvalues of $R$.

Using this decomposition, we define the inverse square root of the correlation matrix:

$$R^{-1/2} = UD^{-1/2}U^T,$$

where

$$D^{-1/2} = \text{diag}\left(\frac{1}{\sqrt{d_1}}, \dots, \frac{1}{\sqrt{d_p}}\right).$$

We define the whitened design matrix as:

$$\widetilde{X} = XR^{-1/2}$$

To understand why this transformation removes correlations, consider the covariance matrix of a row of $X$. Assuming $\text{Cov}(X_i) = R$, we obtain:

$$\begin{aligned}\mathrm{Cov}(\widetilde{X}_i) &= R^{-1/2}\,\mathrm{Cov}(X_i)R^{-1/2} \\ &= R^{-1/2}RR^{-1/2} \\ &= I_p.\end{aligned}$$

Therefore, the transformed predictors have an identity population covariance matrix. This means that the columns of $\widetilde{X}$ are uncorrelated in the population.

Importantly, since our simulated dataset has only $n = 50$ observations, the empirical correlation matrix $\mathrm{cor}(\widetilde{X})$ will not be exactly equal to $I_p$.

Recall our original regression model:

$$y = X\beta + \epsilon.$$

We can insert the identity matrix $R^{-1/2}R^{1/2} = I_p$:

$$\begin{aligned}y &= X\beta + \epsilon \\ &= XR^{-1/2}R^{1/2}\beta + \epsilon \\ &= \widetilde{X}\tilde{\beta} + \epsilon\end{aligned}$$

We therefore define:

$$\widetilde{X} = XR^{-1/2} \qquad \text{and} \qquad \tilde{\beta} = R^{1/2}\beta$$

The error term $\epsilon$ remains unchanged because we are only transforming the predictors and coefficients, not the response variable.

**Important:** Although the original coefficient vector $\beta$ is sparse, the transformed coefficient vector $\tilde{\beta}$ is generally not sparse. This happens because multiplying by $R^{1/2}$ mixes information across coefficients.

Consequently, applying ordinary Lasso directly to $\widetilde{X}$ would not necessarily recover the original sparse coefficient vector. WLasso addresses this by modifying the penalty, which we will derive in the next step.

```
# Eigendecomposition of the correlation matrix
eig <- eigen(R)

U <- eig$vectors
D <- eig$values

# Calculate R^(-1/2)
```

```r
R_inv_sqrt <- U %*%
  diag(1 / sqrt(D)) %*%
  t(U)

# Whiten the predictor matrix
X_tilde <- X %*% R_inv_sqrt

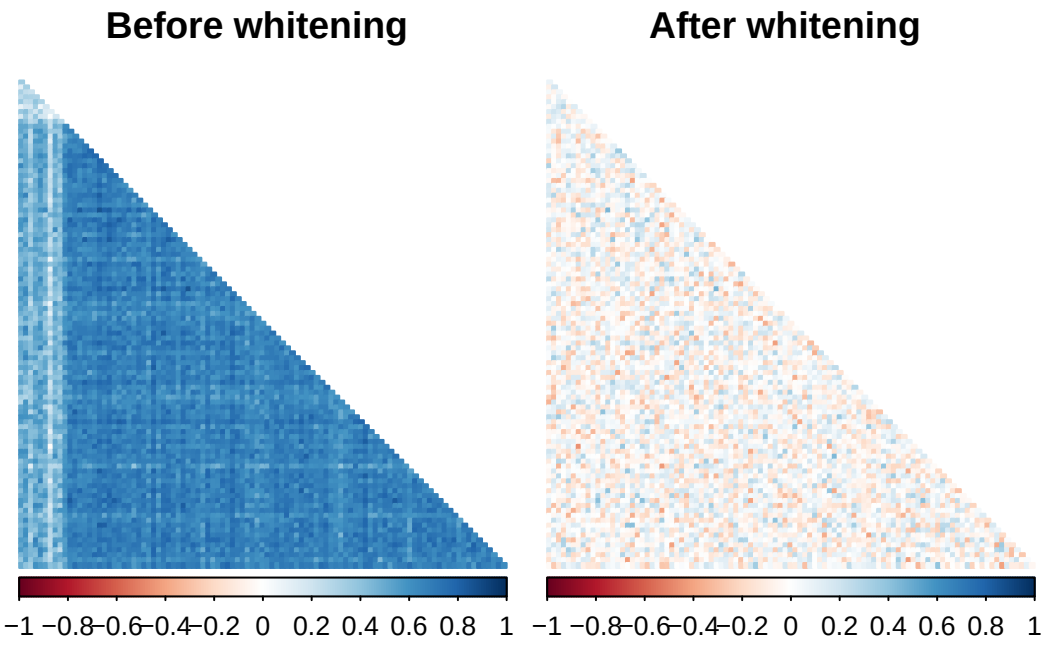
# Check dimensions
dim(X_tilde)
```

```
[1]  50 100
```

```r
par(mfrow = c(1, 2))

corrplot(
  cor(X),
  method = "color",
  type = "lower",
  tl.pos = "n",
  diag = FALSE,
  title = "Before whitening",
  mar = c(0, 0, 2, 0)
)

corrplot(
  cor(X_tilde),
  method = "color",
  type = "lower",
  tl.pos = "n",
  diag = FALSE,
  title = "After whitening",
  mar = c(0, 0, 2, 0)
)
```




```r
par(mfrow = c(1, 1))
```

### 1.5 Deriving the Generalized Lasso Penalty

The classical Lasso minimizes the following objective function:

$$\hat{\beta} = \arg\min_{\beta} \left\{ \|y - X\beta\|_2^2 + \lambda \|\beta\|_1 \right\}.$$

Recall the whitening transformations:

$$\widetilde{X} = XR^{-1/2}, \qquad \tilde{\beta} = R^{1/2}\beta.$$

Rearranging the second transformation gives:

$$\beta = R^{-1/2}\tilde{\beta}.$$

Substituting into the original Lasso objective:

$$\|y - X\beta\|_2^2 + \lambda\|\beta\|_1 = \|y - \widetilde{X}\tilde{\beta}\|_2^2 + \lambda\|R^{-1/2}\tilde{\beta}\|_1.$$

Therefore, the generalized Lasso objective becomes:

$$\boxed{\hat{\tilde{\beta}} = \arg\min_{\tilde{\beta}} \left\{ \|y - \widetilde{X}\tilde{\beta}\|_2^2 + \lambda \|R^{-1/2}\tilde{\beta}\|_1 \right\}}$$

Unlike ordinary Lasso applied to the whitened predictors, the generalized Lasso penalizes a linear transformation of the coefficients.

This is important because $\tilde{\beta}$ is generally not sparse, even when the original $\beta$ is sparse. By penalizing $R^{-1/2}\tilde{\beta} = \beta$, the generalized Lasso encourages sparsity in the original coefficient vector, not the transformed one. We use the package `genlasso` instead of `glmnet` because we penalize a linear transformation of coefficients.

Here each line represents one coefficient as lambda increases.